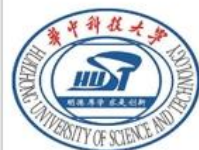


软件工程

第4章 软件设计

4.2 体系结构设计(第1部分：体系结构的初步设计)



内容提要

1.

- 什么是软件体系结构？

2.

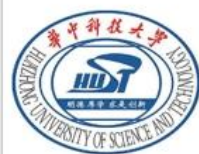
- 如何设计软件体系结构？

3.

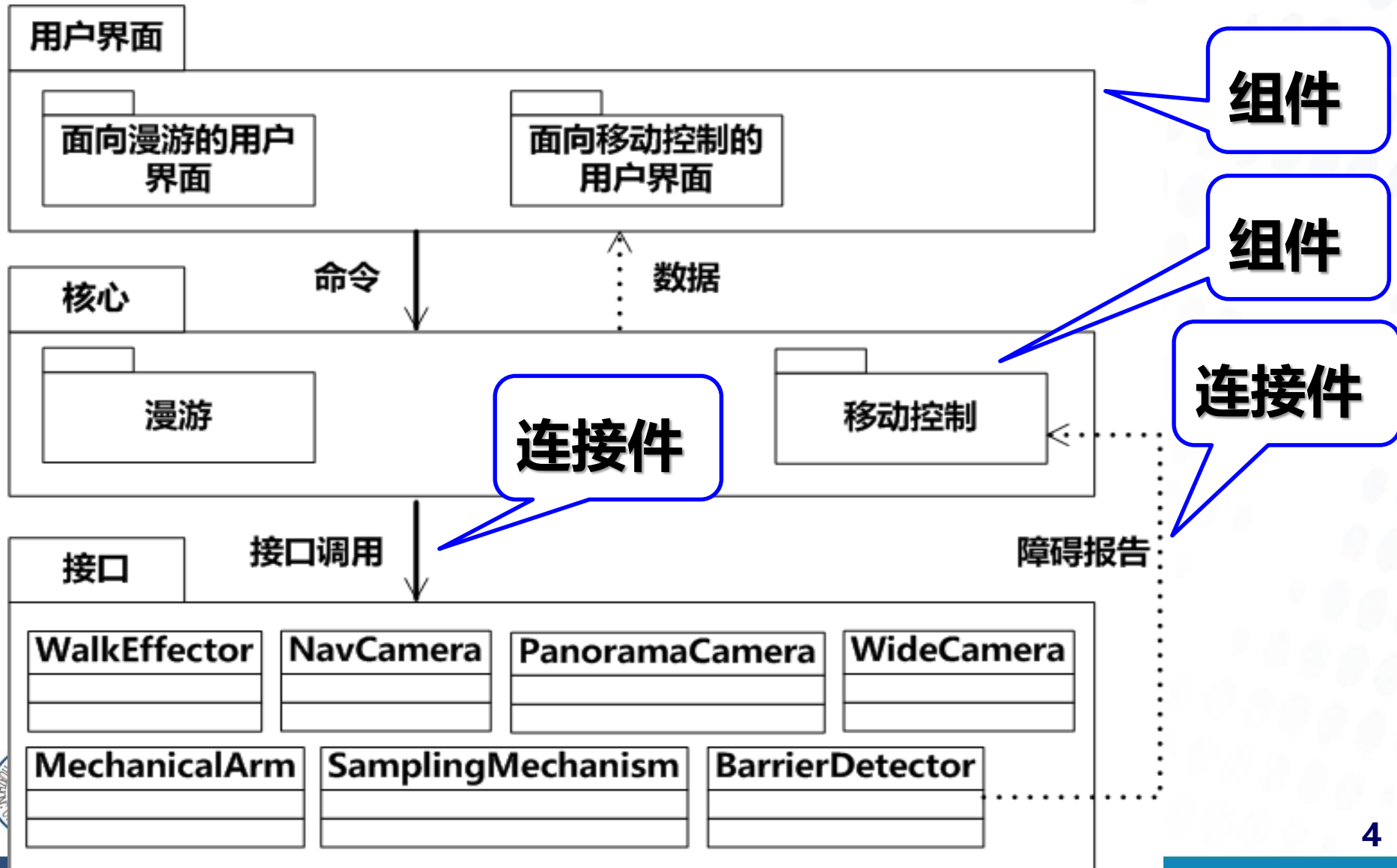
-

严格地，什么是软件体系结构？

- 体系结构包括**组件** (component)、**连接件** (connector) 和**约束** (constraints) 三大要素
 - ◆ 组件即“软件设计元素”（子系统、构件或类）
 - ◆ 连接件表示组件之间的连接和交互关系
 - ◆ 约束表示组件中的元素应满足的条件，以及组件经由 连接件 组装成更大组件时应满足的条件



软件体系结构示例



软件体系结构的约束示例

- 位于较高层次的设计元素可以向低层元素发送命令，低层元素只能向高层传递数据或通告，反之不行
- 每个软件元素根据其职责位于最恰当的层次，不可错置
 - ◆ 例如，核心层不能包含界面呈现和用户输入接收职责，也不能直接与外部实体交互

内容提要

1.

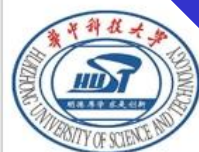
- 什么是软件体系结构？

2.

- 如何设计软件体系结构？

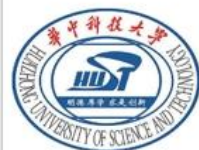
3.

-



首先，体系结构设计任务是什么？

- 建立满足软件需求的**软件高层结构**，在其中设置软件系统中所有重要的模块并明确定义它们的**职责划分**及**协作关系**
 - ◆ 针对同一需求模型，目标软件系统中模块的设置、职责划分和协作方式可以多种多样
 - ◆ 体系结构设计必须追求在正确性、可扩展性、可伸缩性、可行性等方面整体表现最优的软件体系结构



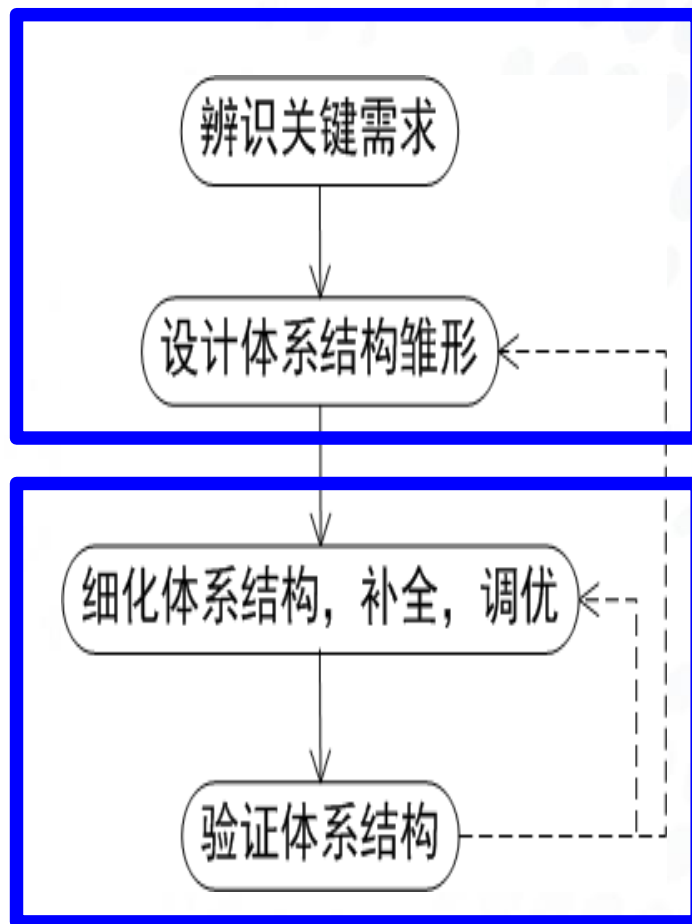
大型软件的体系结构设计很难一蹴而就

□ 初步设计阶段：

- ◆ 仅针对需求模型中的**关键需求项**来构思软件体系结构的**雏形**

□ 精化阶段：

- ◆ 全面地考虑所有的软件需求项，**丰富**体系结构雏形的细节
- ◆ 对体系结构进行**调优**和**验证**



什么是“关键需求项”？

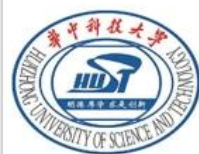
- 关键需求项：对用户满意度及软件结构均有重要影响的需求项
- 为什么不在初步设计阶段考虑所有需求项？
 - ◆ 不能奢望在启动体系结构设计之前拥有一个稳定的需求项全集
 - ◆ 即便拥有，也没有充裕的时间深入研究所有的需求项——“与其伤其十指，不如断其一指”



因此，正确的体系结构设计策略只能是...

□ 基于关键需求项推导体系结构的雏形，其他需求项用于补全、细化、精化

◆ “20-80规则”：20%的关键需求项决定80%的体系结构形态



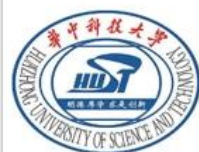
什么是“体系结构雏形”？

□ 软件体系结构的初始形态

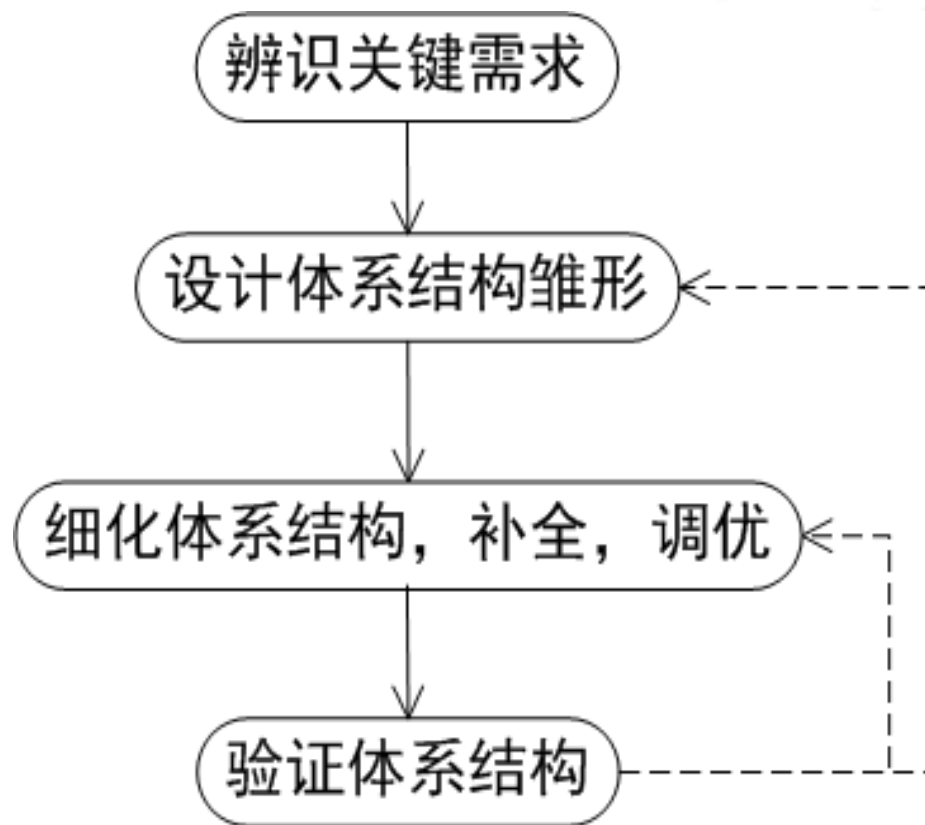
- ◆ 相对于后续的精化的体系结构形态而言，雏形更粗略、更容易构建

□ 为什么不直接构建软件体系结构的精化形态？

- ◆ 先集中精力解决主要矛盾
- ◆ 勾勒出若干体系结构雏形进行比较和权衡 较容易、更节约



回望体系结构设计的过程模型



为什么要验证精化后的体系结构？

软件体系结构的验证

- 验证活动必须回答以下问题：
 - ◆ 体系结构是否足以支撑所有软件需求项的实现？
 - ◆ 是否还有进一步优化的空间？
- 为什么在采纳体系结构之前必须验证？
 - ◆ 体系结构对目标软件质量的影响甚巨
 - ◆ 如果在软件实现过程中发现软件体系结构存在缺陷，返工的代价将非常大

小结：如何设计软件体系结构？

辨识关键需求(关键的功能需求和关键的质量需求)

创建体系结构雏形

精化体系结构

验证体系结构

如何辨识关键的功能需求? (1/3)

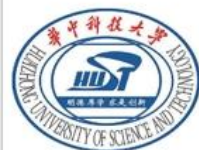
□ 确定基础性功能

- ◆ 其他功能依赖于这些功能, 或仅当这些功能实现之后其他功能才有意义

□ 确定重要性明显高于其他功能的核心功能

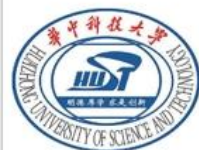
□ 标识实现难度较大、实现风险较高的功能

- ◆ 体系结构设计必须尽早探索高难度、高风险功能的设计方案



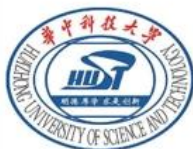
如何辨识关键的功能需求(2/3)

- 标识最能体现待开发软件**特色**的功能需求
- 确定前述类别没有覆盖，但对**用户满意度**影响甚巨的功能需求
- 确定为实现**对外接口**所必需的支持功能
 - ◆ 对外接口的实现依赖于这些功能需求项的实现



如何辨识关键的功能需求(3/3)

- 剔除对体系结构塑形无贡献的功能需求项
 - ◆ 对功能需求项A和B，如果职责集 $R(A) \supseteq R(B)$ ，且职责间协作关系 $C(A)$ 类似于 $C(B)$ ，那么，在A加入关键集之后，应将B剔除

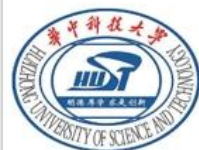


案例分析：MCS中的关键功能需求项



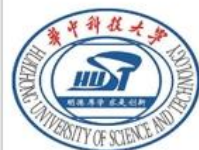
接下来，如何辨识关键的质量需求？

- 区分对**用户满意度**影响甚巨的质量需求与 “锦上添花” 型的质量需求
- 标识实现**难度较大**、**实现风险较高**的质量需求
 - ◆ 并非所有高风险、高难度的质量需求项均须纳入关键需求集
 - ◆ 设计师必须决定，在给定的时间和成本约束前提下重点支持哪些质量需求项



如何辩识关键的质量需求(续)

- 针对所有可能冲突的质量需求项对(例如 性能需求与可扩展性需求), 确定并明示权衡决策, 然后决定采纳以下选项之一:
 - ◆ 仅将优先支持的质量需求项纳入关键集
 - ◆ 将二者均纳入; 或者
 - ◆ 均不纳入关键集



最后，如何辩识关键的约束性需求？

- 只要某项约束对软件结构有影响，就必须纳入关键集
 - ◆ 与软件体系结构无关的约束，例如编程语言限制等，不应进入关键集

辨识关键需求项的经验规则

- 关键质量需求项所占比例应大于关键功能占比
 - ◆ 因为质量需求对体系结构初始塑形的影响力比功能需求更大
- 在功能需求项和质量需求项较少时关键需求项占比应分别加大，否则应相应地缩小



案例分析：MCS中关键的非功能需求项

□ 性能需求：

哪些需求项应进入关键集？

- ◆ 在仿真过程中月球车每移动一步的完成时间小于1秒

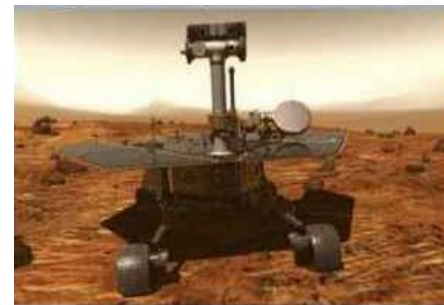
□ 可扩展性需求：

- ◆ 目前只需支持观察者通过单机界面查看漫游过程，但未来必须支持大量爱好者通过Internet和智能手机查看漫游过程

□ 可替换性需求：

- ◆ 要求与MCS交互的设备和障碍探测软件

可替换，以支持MCS复用于未来的火星探测计划



案例分析：MCS中关键的非功能需求项

□ 性能需求：

为什么？

- ◆ 在仿真过程中月球车每移动一步的完成时间小于1秒

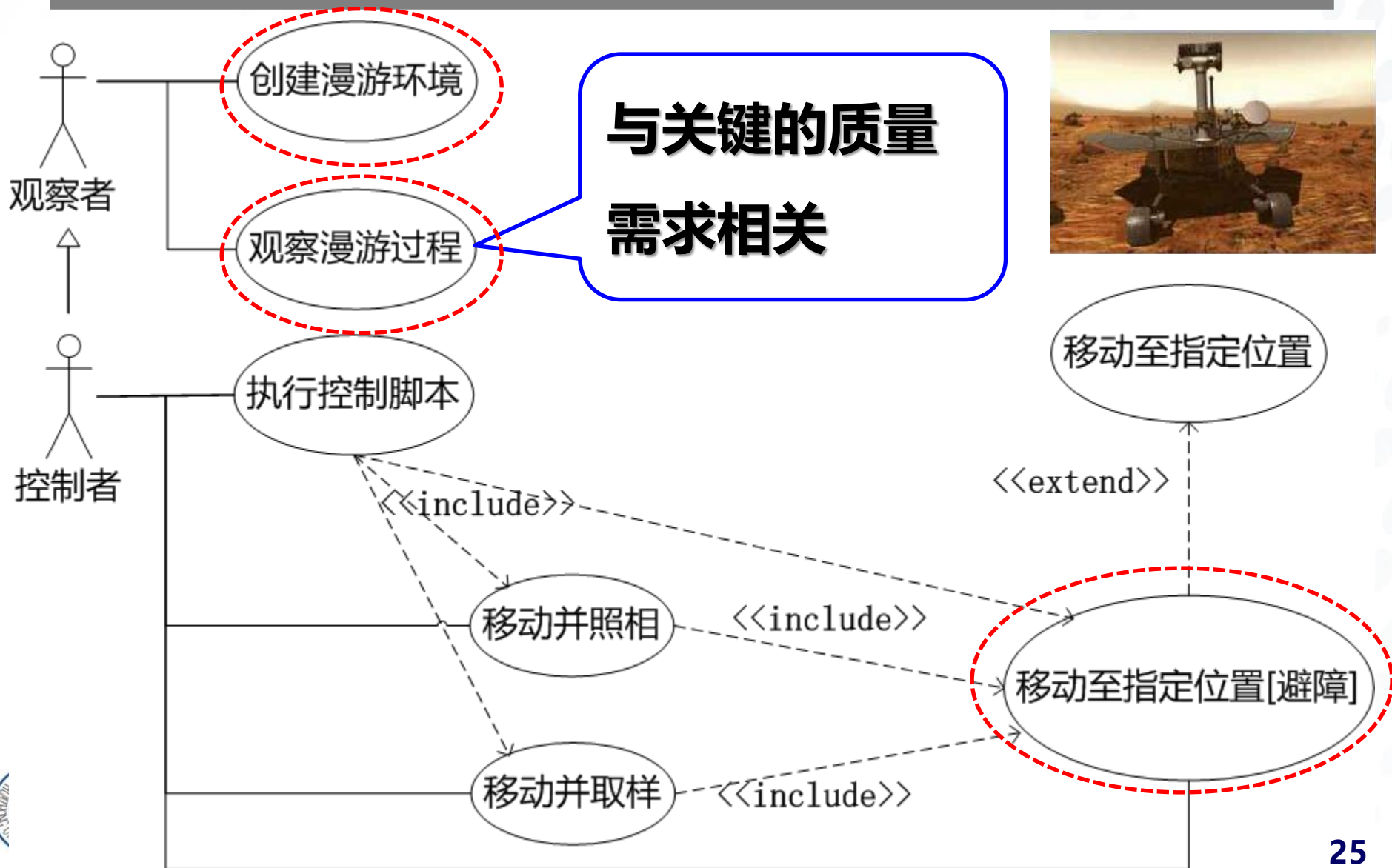
□ 可扩展性需求：

- ◆ 目前只需支持观察者通过单机界面查看漫游过程，但未来必须支持大量爱好者通过Internet和智能手机查看漫游过程

□ 可替换性需求：

- ◆ 要求与MCS交互的设备和障碍探测软件可替换，以支持MCS复用于未来的火星探测计划

注意：MCS的扩展性需求涉及“观察漫游过程”



总结：如何设计软件体系结构？

辨识关键需求(关键的功能需求和关键的质量需求)

创建体系结构雏形

精化体系结构

且听下回分解

验证体系结构